AI Mid-Term Prep: Search Algorithms

BFS, DFS, UCS, Heuristic Search and A*: theory, worked traces, practice problems and a cheat sheet

1. What the paper will look like

Your sir said: no programming, only 3 problems, and only students who understand the concept can solve them. That means hand-trace problems on small graphs, trees or puzzles. The list below is my inference from the syllabus, not something confirmed, so prepare all types.

#	Likely question type	What they ask	Likelihood
1	Trace BFS / DFS on a tree or graph	Order of expansion, frontier contents, path found, nodes expanded	Very high
2	UCS on a weighted graph	Priority-queue trace, final path and cost, compare with BFS	Very high
3	Greedy / A* with a heuristic table	f = g + h trace, path, cost, nodes expanded, compare Greedy vs A*	Very high
4	Heuristic properties	Is h admissible? consistent? which heuristic dominates? design one for 8-puzzle or grid	High
5	Choose / compare algorithms	Why BFS fails on weighted graphs, why DFS is not optimal, space vs time trade-offs	High
6	Problem formulation	States, actions, goal test, path cost for jugs, puzzle, maze	Medium
7	Short proofs	A* optimality, consistency implies admissibility	Medium

With 5 topics and 3 problems, a likely split is: one on BFS/DFS, one on UCS, one on Greedy/A*. Do not bet on this. The same trace skill covers all of them.

2. The one framework behind every algorithm

A search problem has 5 parts: initial state, actions, transition model, goal test, path cost.

Every algorithm in your syllabus runs the same loop. Only the order in which the frontier is served changes.

1. Put the start node in the frontier.
2. Pick a node from the frontier (the algorithm decides which).
3. If it passes the goal test, return its path.
4. Otherwise expand it: generate its children, add the new ones to the frontier, and mark the node as explored.
5. Repeat. If the frontier is empty, return failure.

Algorithm	Frontier type	Picks node with	Uses
BFS	FIFO queue	Shallowest (oldest)	depth only
DFS	LIFO stack	Deepest (newest)	depth only
UCS	Priority queue	Smallest g(n)	path cost so far
Greedy best-first	Priority queue	Smallest h(n)	heuristic only
A*	Priority queue	Smallest f(n) = g(n) + h(n)	both

Three identities worth knowing, because examiners love them:

- BFS is UCS when every step costs 1.
- UCS is A* with h(n) = 0.
- Greedy is A* that ignores g(n).

Evaluation criteria and notation

- Completeness: does it always find a solution if one exists?
- Optimality: is the solution it returns the cheapest?
- b = branching factor, d = depth of the shallowest goal, m = maximum depth of the tree, C* = optimal solution cost, ε = minimum step cost.

Conventions you must state in the exam

Most lost marks come from unstated conventions, not from wrong algorithms. At the top of every answer, write your assumptions:

- Tie-breaking: alphabetical order (or left-to-right) unless the question says otherwise.
- Goal test is applied when a node is expanded (popped). AIMA tests BFS at generation time, which saves some expansions. If sir's notes do it differently, follow sir.
- Explored set is used (graph search): never re-expand an explored node.
- Expanded means popped and its children generated. Generated means created and added to the frontier. Do not mix them up when counting.

3. Uninformed search

3.1 Breadth-First Search (BFS)

- Expands all nodes at depth k before depth k+1. Frontier is a FIFO queue.
- Complete if b is finite. Optimal only if all step costs are equal (finds the shallowest goal, not the cheapest).
- Time O(b^d), space O(b^d). Space is the real killer: it must store the whole frontier.
- Common mistake: calling BFS optimal on a weighted graph. It minimises the number of edges, not the cost.

3.2 Depth-First Search (DFS)

- Always expands the deepest node in the frontier. Frontier is a LIFO stack (or recursion).
- Tree search: not complete (infinite depth or loops), not optimal. Graph search with an explored set: complete on finite graphs, still not optimal.
- Time O(b^m), space O(b·m) for tree search. This linear space is the main reason to use it.
- Note the precise claim. 'DFS is not complete' is only true for infinite state spaces or tree search with cycles. Writing that distinction earns full marks.
- Order trap: to explore the alphabetically first child first, push children onto the stack in reverse order. Recursive DFS does this naturally.

3.3 Uniform-Cost Search (UCS)

- Expands the node with the smallest path cost g(n). Frontier is a priority queue ordered by g.
- Complete and optimal if every step cost is at least ε > 0. Time and space O(b^(1 + floor(C*/ε))).
- Goal test must happen when the goal is popped, not when it is generated. A goal can be generated first via an expensive path, and a cheaper path may still sit in the frontier.
- If a node is reached again with a lower g, update its cost and parent in the frontier. Do not keep the old, worse entry.
- Explores in contours of equal cost, with no sense of direction toward the goal. That is exactly the weakness A* fixes.

4. Informed (heuristic) search

A heuristic h(n) estimates the cost from n to the nearest goal. It encodes problem knowledge. Always h(goal) = 0.

4.1 Greedy best-first search

- Expands the node with the smallest h(n). Ignores the cost already paid.
- Not optimal, and not complete in tree search (can loop). Time and space O(b^m) in the worst case, but often fast in practice.

4.2 A* search

- f(n) = g(n) + h(n): cost so far plus estimated cost to go. Expand the smallest f.
- Complete when b is finite and costs are at least ε.
- Optimal if h is admissible (tree search) or consistent (graph search).
- Optimally efficient: no other algorithm using the same heuristic is guaranteed to expand fewer nodes.
- Main weakness: memory. It keeps all generated nodes, which is exponential. Fixes: IDA*, SMA* (name them, no detail needed).

4.3 Admissible and consistent heuristics

- Admissible: 0 <= h(n) <= h*(n) for every n, where h*(n) is the true cheapest cost to the goal. It never overestimates.

- Consistent (monotone): h(n) <= c(n, a, n′) + h(n′) for every edge. A triangle inequality.
- Consistent implies admissible. The reverse is false.
- If h is consistent, f never decreases along a path, the first time A* expands a node it already has the optimal g, and no node ever needs re-opening.
- Dominance: if h2(n) >= h1(n) for all n and both are admissible, h2 dominates h1, and A* with h2 expands no more nodes than with h1 (up to ties). Bigger admissible heuristic means better.
- Creating heuristics: solve a relaxed problem (remove constraints) exactly. Misplaced tiles relaxes 'tiles move to any square'. Manhattan distance relaxes 'tiles move through occupied squares'.
- Checking admissibility in an exam: compute the true remaining cost h*(n) for each node and compare with h(n). Checking consistency: test every edge, both directions if undirected.

5. Master comparison

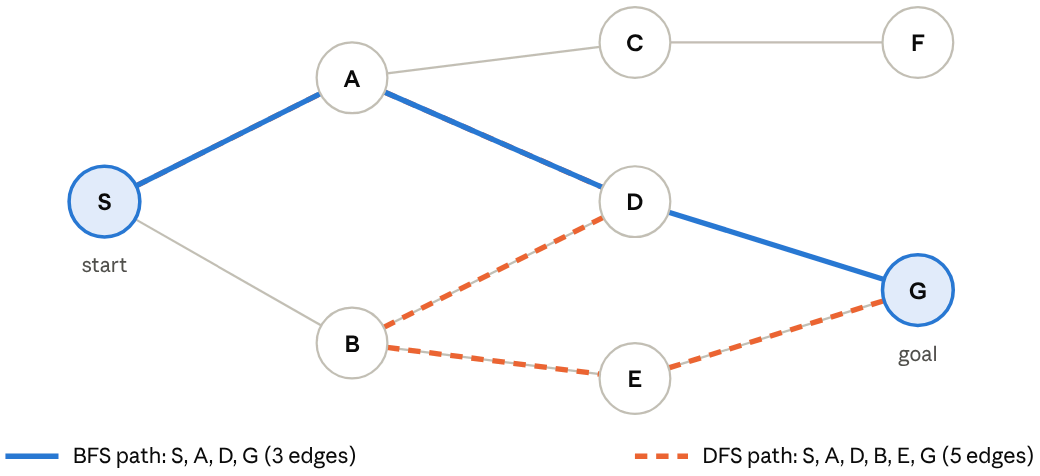
	BFS	DFS	UCS	Greedy	A*
Complete	Yes (finite b)	No in general; yes on finite graph with explored set	Yes (cost >= ε)	No (tree search)	Yes (cost >= ε)
Optimal	Only if unit cost	No	Yes	No	Yes, if h admissible/consistent
Time	O(b^d)	O(b^m)	O(b^(1+C*/ε))	O(b^m)	Exponential, depends on h
Space	O(b^d)	O(b·m)	O(b^(1+C*/ε))	O(b^m)	Exponential (keeps all nodes)
Uses cost g	No	No	Yes	No	Yes
Uses heuristic h	No	No	No	Yes	Yes

6. Worked Problem 1: BFS and DFS

Graph (undirected, edges unweighted). Start S, goal G. Break ties alphabetically.

Node	Neighbours
S	A, B
A	S, C, D
B	S, D, E
C	A, F
D	A, B, G
E	B, G
F	C
G	D, E

BFS finds a 3-edge path; DFS returns a 5-edge path



Problem 1 graph · BFS and DFS paths

The solid line is the BFS path and the dashed line is the DFS path; both start at S and end at G.

BFS trace (FIFO queue)

Step	Expand	Frontier after expansion (front first)	Note
1	S	A, B	

Step	Expand	Frontier after expansion (front first)	Note
2	A	B, C, D	S already explored
3	B	C, D, E	D already in frontier, not added again
4	C	D, E, F	
5	D	E, F, G	G generated here
6	E	F, G	G already in frontier
7	F	G	
8	G	empty	goal test passes

Expansion order: S, A, B, C, D, E, F, G. Path: S, A, D, G (3 edges, shortest). If the goal test is done at generation, BFS stops at step 5 when G is generated, after only 5 expansions. State which convention you use.

DFS trace (LIFO stack, top listed first)

Step	Expand	Stack after expansion (top first)	Note
1	S	A, B	A on top so it is explored first
2	A	C, D, B	
3	C	F, D, B	
4	F	D, B	dead end, backtrack
5	D	B, G, B	B is pushed again; the old B stays lower in the stack
6	B	E, G, B	S, D explored
7	E	G, G, B	
8	G		goal

Expansion order: S, A, C, F, D, B, E, G. Path: S, A, D, B, E, G (5 edges). It reached the goal but the path is longer than BFS's 3 edges. That is the standard proof that DFS is not optimal. The leftover B in the stack is simply skipped if popped later, because it is already explored.

What to write as the conclusion: BFS gives the shortest path (3 edges) but needs a larger frontier. DFS needs less memory but returned a path 2 edges longer.

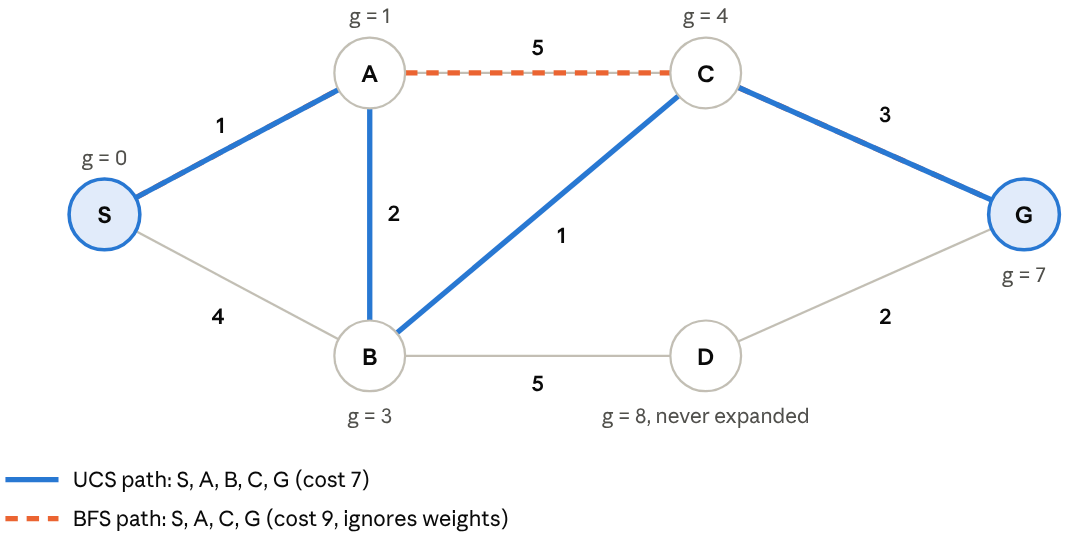
7. Worked Problem 2: Uniform-Cost Search

Undirected weighted graph. Start S, goal G.

Edge	Cost
S - A	1
S - B	4
A - B	2
A - C	5
B - C	1
B - D	5
C - G	3
D - G	2

UCS finds cost 7; BFS ignores weights and pays 9

Numbers on edges are step costs; g is the cheapest cost found so far for each node



Problem 2 graph · UCS and BFS paths with g values

The solid line is the cheapest path (UCS); the dashed line is what BFS returns when it ignores weights.

Notation: node(g) in the frontier.

Step	Pop	What happens	Frontier after
1	S(0)	generate A(1), B(4)	A(1), B(4)
2	A(1)	B via A costs 1+2 = 3, better than 4, so update B to 3 (parent A). C = 1+5 = 6	B(3), C(6)
3	B(3)	C via B costs 3+1 = 4, better than 6, so update C to 4 (parent B). D = 3+5 = 8	C(4), D(8)
4	C(4)	G = 4+3 = 7	G(7), D(8)
5	G(7)	goal popped	

Expansion order: S, A, B, C, G. Path: S, A, B, C, G with cost 7. Node D was never expanded.

Compare with BFS on the same graph. BFS ignores weights and returns S, A, C, G (3 edges) with cost 1+5+3 = 9. UCS returns cost 7. This comparison is a very natural exam follow-up.

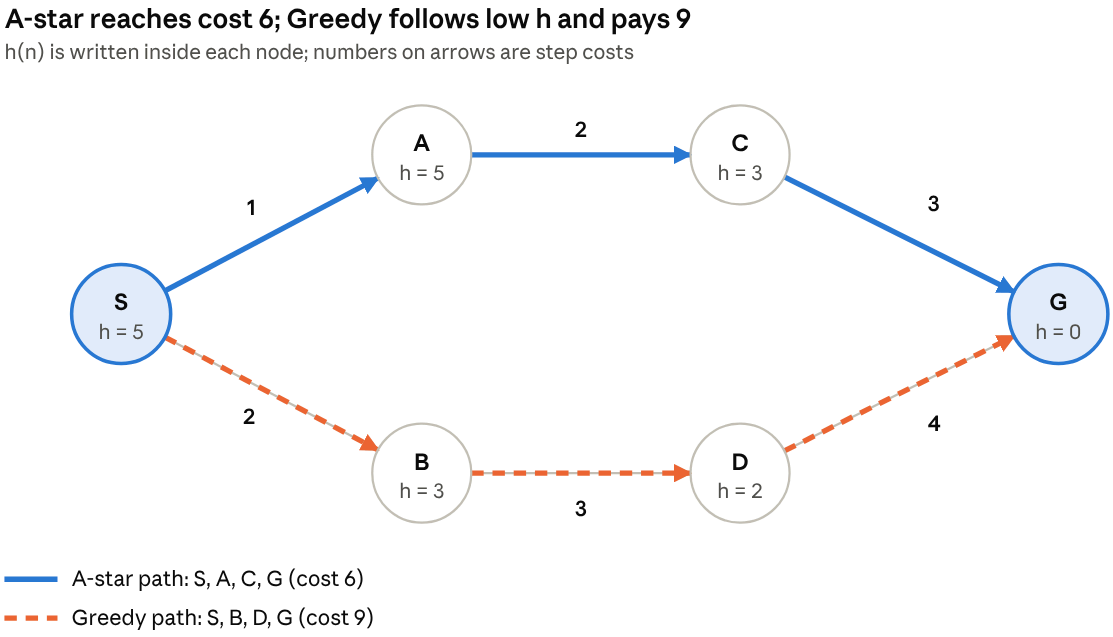
Why you must test the goal at pop time: if the cheapest route to G had been found after G was first generated, stopping at generation would return a worse path. UCS only guarantees optimality when the goal is the cheapest item in the priority queue.

8. Worked Problem 3: Greedy vs A*

Directed graph. Start S, goal G.

Edge	Cost
S to A	1
S to B	2
A to C	2
B to D	3
C to G	3
D to G	4

Heuristic: h(S) = 5, h(A) = 5, h(B) = 3, h(C) = 3, h(D) = 2, h(G) = 0.



Problem 3 graph · A-star and Greedy paths with h values

The solid line is the A-star path and the dashed line is the Greedy path.

Check first that h is admissible. True costs to G: S = 6, A = 5, B = 7, C = 3, D = 4. Every h is at most the true cost, so it is admissible. It is also consistent, for example S to B: 5 <= 2 + 3, and A to C: 5 <= 2 + 3.

Greedy best-first

Expand S: A has h = 5, B has h = 3. Pick B. Expand B: D has h = 2. Pick D. Expand D: G. Done. Path: S, B, D, G with cost 2+3+4 = 9. Expanded: S, B, D (then G). Greedy rushed toward low h and ignored cost. The optimal cost is 6.

A* (f = g + h)

Step	Pop	Children generated (g, h, f)	Frontier (f)
1	S (0+5 = 5)	A (1, 5, 6), B (2, 3, 5)	B(5), A(6)
2	B	D (5, 2, 7)	A(6), D(7)
3	A	C (3, 3, 6)	C(6), D(7)
4	C	G (6, 0, 6)	G(6), D(7)
5	G	goal	

Path: S, A, C, G with cost 6, which is optimal. Expansion order: S, B, A, C, G. A* briefly tried B (f = 5 looked attractive) but never expanded D, because f(D) = 7 stayed above the optimal cost 6.

What breaks if h is inadmissible

Suppose h(C) = 8 instead of 3 (true cost is 3, so it overestimates). Then f(C) = 3 + 8 = 11. A* expands S, B, A, then D (f = 7), generates G with f = 9, and pops G before C. It returns S, B, D, G with cost 9, which is not optimal. This is the standard demonstration that admissibility is required for optimality.

9. Worked Problem 4: 8-puzzle with A*

The blank is written as 0 (no tile uses that number). Rows are separated by /.

Start: 283/164/705. Goal: 123/804/765.

Two heuristics:

- h1 = number of misplaced tiles (blank not counted).
- h2 = sum of Manhattan distances of tiles to their goal squares.

For the start state: misplaced tiles are 2, 8, 1, 6, so h1 = 4. Manhattan distances: tile 2 is 1 away, tile 8 is 2 away, tile 1 is 1 away, tile 6 is 1 away, so h2 = 5. Here h2 >= h1, so h2 dominates h1. Both are admissible: each move changes one tile's position by one square, so h2 never overestimates. The true optimal length is 5.

A* trace with h2 (moves that undo the previous move are omitted)

Expanded state	g	h2	f	Children generated (state, g, h2, f)
283/164/705	0	5	5	283/104/765 (1,4,5); 283/164/075 (1,6,7); 283/164/750 (1,6,7)
283/104/765	1	4	5	203/184/765 (2,3,5); 283/014/765 (2,5,7); 283/140/765 (2,5,7)
203/184/765	2	3	5	023/184/765 (3,2,5); 230/184/765 (3,4,7)
023/184/765	3	2	5	123/084/765 (4,1,5)
123/084/765	4	1	5	123/804/765 (5,0,5) = goal

Solution: blank moves Up, Up, Left, Down, Right (5 moves). f stayed at 5 the whole time, which means h2 was exactly right along the optimal path. Nodes with f = 7 were generated but never expanded. With h1 the same problem would expand more nodes, which is what dominance predicts.

10. Theory questions with model answers

Q1. Why is BFS not optimal for weighted graphs? It orders nodes by depth (number of edges) and ignores edge costs. It returns the shallowest goal, which may be expensive. Example: Problem 2, BFS gave cost 9 versus UCS cost 7.

Q2. When is BFS optimal? When path cost is a non-decreasing function of depth, for example all step costs equal.

Q3. Why does UCS test the goal on expansion? The first time the goal is generated may be via an expensive path while a cheaper path is still in the frontier. The goal is guaranteed cheapest only when it is the minimum-g node popped.

Q4. Prove that A-star with admissible h is optimal (tree search, sketch). Suppose a suboptimal goal G2 is in the frontier. Let n be an unexpanded node on an optimal path to the optimal goal G. Then f(n) = g(n) + h(n) <= g(n) + h*(n) = C* (admissibility). Also f(G2) = g(G2) > C*, since G2 is suboptimal and h(G2) = 0. So f(n) < f(G2), and n is expanded before G2. Hence G2 is never selected before G is found.

Q5. Prove consistency implies admissibility (sketch). Take an optimal path n = n0, n1, ..., nk = goal. By induction from the goal: h(nk) = 0, and h(n_i) <= c(n_i, n_{i+1}) + h(n_{i+1}). Summing along the path gives h(n) <= total path cost = h*(n).

Q6. What is the difference between Greedy and A-star? Greedy uses h only, so it is fast but neither optimal nor complete in tree search. A* uses g + h, so it is optimal with an admissible or consistent h, at the cost of exponential memory.

Q7. Why is A-star with a better heuristic faster? The A* expanded set is every node with f(n) < C*. A larger admissible h raises f values, so fewer nodes fall below C*.

Q8. Tree search versus graph search? Tree search may re-expand states reached by different paths and can loop forever. Graph search keeps an explored set to avoid this, but for A* it requires consistent h (or re-opening of nodes) to stay optimal.

Q9. DFS versus BFS memory? DFS stores one path plus siblings: O(b·m). BFS stores a whole level: O(b^d). That is why DFS is preferred when the state space is huge and any solution is acceptable.

Q10. What is the weakness of A-star? Memory. It stores all generated nodes. Memory-bounded variants: IDA*, SMA*, RBFS.

11. Problem formulation (they may ask this too)

Example: 4-litre and 3-litre jugs, goal is exactly 2 litres in the 4-litre jug.

- State: (x, y) = litres in the 4L jug and the 3L jug, with 0 <= x <= 4 and 0 <= y <= 3.
- Initial state: (0, 0).
- Actions: fill either jug, empty either jug, pour one into the other until the source is empty or the target is full.
- Transition model: for example pour 3L into 4L from (x, y) gives (min(4, x+y), y - (min(4, x+y) - x)).
- Goal test: x = 2.
- Path cost: 1 per action. Since all costs are equal, BFS finds the fewest-step solution.

Rule of thumb for any formulation: a state must contain exactly the information needed to decide the legal moves and the goal test, and nothing more.

12. Practice problems (try them first, then check answers)

P1. BFS vs DFS on a tree. Tree: A has children B, C, D. B has children E, F. C has child G. D has children H, I. Goal G, alphabetical order. Give expansion order and nodes expanded for both.

P2. UCS. Directed edges: S to A (4), S to B (1), B to A (2), B to C (6), A to G (6), C to G (1). Trace UCS from S to G.

P3. Greedy vs A-star. Directed edges: S to A (1), S to B (4), A to B (2), A to C (5), B to G (3), C to G (2). Heuristic: h(S) = 5, h(A) = 4, h(B) = 3, h(C) = 2, h(G) = 0. Trace Greedy and A*. Is h consistent?

P4. 8-puzzle heuristics. Start 103/425/786, goal 123/456/780 (0 is the blank). Compute h1 and h2, and find the optimal solution.

Answers

P1. BFS expands A, B, C, D, E, F, G: 7 nodes. DFS expands A, B, E, F, C, G: 6 nodes. Path A, C, G for both. DFS happens to win here because the goal is in the left part of the tree. Changing the goal to D would reverse the result, so never claim one is always faster.

P2. S(0) generates A(4), B(1). Pop B(1): A via B = 3 (update), C = 7. Pop A(3): G = 9. Pop C(7): G via C = 8 (update from 9). Pop G(8). Expansion order S, B, A, C, G. Path S, B, C, G, cost 8. The alternative S, B, A, G costs 9.

P3. Greedy: from S, B (h = 3) beats A (h = 4). Expand B, reach G. Path S, B, G cost 7. A*: S(f = 5) generates A (1+4 = 5), B (4+3 = 7). Pop A: B via A has g = 3, which is better than 4, so update B to f = 6. C (6+2 = 8). Pop B(6): G (g = 6, f = 6). Pop G. Path S, A, B, G cost 6, optimal. Greedy gave 7. Consistency check on every edge: S to A: 5 <= 1+4. S to B: 5 <= 4+3. A to B: 4 <= 2+3. A to C: 4 <= 5+2. B to G: 3 <= 3. C to G: 2 <= 2. All hold, so h is consistent.

P4. Misplaced tiles: 2, 5, 6, so h1 = 3. Manhattan: tile 2 is 1 away, tile 5 is 1 away, tile 6 is 1 away, so h2 = 3. Optimal solution is 3 moves: blank down, blank right, blank down. Here h1 = h2 = h*, so A* expands only 3 nodes.

13. Exam-day strategy

1. Write assumptions first: tie-break, goal test timing, explored set. One line, saves marks if your order differs from the examiner's.
2. Always use a trace table: columns for step, node expanded, frontier with costs (g, or h, or f), explored set. Examiners award method marks per row.
3. Draw the search tree briefly when asked for it, labelling nodes with order of expansion. Do not skip the final path and total cost.
4. For A*, write g, h and f for every generated node. A single wrong g propagates, so recompute when a node is reached twice.
5. State the final answer clearly: path, cost, nodes expanded, and whether it is optimal and why.
6. If a result is surprising (DFS finding a long path, Greedy failing), say why. That one sentence is what separates concept from memorisation.

Common mistakes

- Stopping UCS or A* when the goal is generated instead of popped.
- Forgetting to update a frontier node when a cheaper path to it is found.
- Using the heuristic in UCS or BFS ordering.
- Claiming A* is always optimal without mentioning admissible or consistent h.
- Confusing expanded count with generated count.
- Adding already explored nodes back into the frontier.
- Inconsistent tie-breaking mid-trace.

14. One-page revision cheat sheet

- BFS: queue, shallowest first, complete, optimal only for unit cost, space O(b^d).
- DFS: stack, deepest first, not optimal, space O(b·m), complete only with finite graph plus explored set.
- UCS: priority by g, complete and optimal if cost >= ε, goal test on pop, update frontier costs.
- Greedy: priority by h, not optimal, not complete in tree search.
- A*: priority by g + h, optimal if h admissible (tree) or consistent (graph), memory heavy.
- Admissible: h <= h*. Consistent: h(n) <= c(n, n′) + h(n′). Consistent implies admissible.

- Dominance: larger admissible h means fewer expansions.
- BFS = UCS with unit costs. UCS = A* with h = 0. Greedy = A* ignoring g.
- Heuristics come from relaxed problems: misplaced tiles, Manhattan distance.
- Revision plan: day 1 theory plus Problems 1 and 2, day 2 Problems 3 and 4 plus theory Q&A, day 3 redo P1 to P4 from a blank page and time yourself. Aim for under 12 minutes per problem.